Project - High Level Design
on
sports-content-generator

Course Name: generative-AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	ABHAY SINGH THAKUR	EN22CS304003
2	SARTHAK GOKHALE	EN22CS304057
3	HIMANSHI PORWAL	EN22CS306021
4	AKASH MUVEL	EN23EE3L1001
5	PRATHAM JOSHI	EN22CS306037

Group Name: ***Group 04D11***

*Project Number:****GAI-30***

Industry Mentor Name: ***Mr. Yash***

*University Mentor Name:****Dr Hemlata patel***

*Academic Year:****2022-2026***

1. Introduction

1.1 Scope of the Document

This High-Level Design document presents the overall architecture and working of the Sports_GenAI system. It explains the major modules, their interactions, the data flow across the application, the external and internal interfaces, and the high-level technical decisions taken during the project design.

The purpose of this document is to provide a clear system-level understanding of how the application accepts user inputs, processes them through backend services, enhances the request using context retrieval, and generates sports-related content using a generative AI model. This document focuses on the structure and behaviour of the full system rather than source-code-level implementation details.

1.2 Intended Audience

This document is intended for:

- faculty evaluators who need a structured understanding of the project.
- project mentors and reviewers assessing the technical design.
- developers who may extend or maintain the system.
- testers who need to understand the end-to-end flow.
- students and team members who want a system overview before implementation or presentation.

1.3 System Overview

Sports_GenAI is a web-based generative AI application designed to create sports content automatically from structured user inputs. The application allows users to enter details such as sport type, content type, match title, key highlights, tone, and desired content length. Based on this input, the system generates a sports article, summary, report, or blog in a natural-language format.

At a high level, the system works as follows:

- the user interacts with a web dashboard.
- the frontend collects and validates input.
- the request is sent to the FastAPI backend.
- the backend retrieves relevant contextual information from a vector database.
- a prompt is constructed using the user input and retrieved context.
- the prompt is passed to the LLM service.
- the generated article is returned to the frontend and displayed to the user.

2. System Design

2.1 Application Design

The Sports_GenAI application follows a layered and modular architecture. This design improves maintainability, scalability, and separation of concerns.

Presentation Layer

The presentation layer is the frontend built with React and Vite. It provides the user interface through which a user can enter content-generation parameters and view the generated result. This layer is responsible for:

- rendering the dashboard
- collecting user inputs through forms
- validating mandatory fields at the client side
- sending requests to the backend
- displaying generated content and error messages
- managing theme or UI preferences if applicable

Application Layer

The application layer is implemented using FastAPI. It exposes API endpoints for content generation and health monitoring. This layer acts as the entry point for all backend operations. Its responsibilities include:

- receiving the request from the frontend
- validating incoming request payloads
- orchestrating the flow between services
- formatting the response
- handling exceptions and returning appropriate HTTP responses

Service Layer

The service layer contains the main business logic of the system. According to the uploaded LLD, the main services are prompt_builder.py, vector_service.py, and llm_service.py.

- converting structured inputs into an effective AI prompt
- retrieving similar sports-related context from the vector store
- invoking the language model for final content generation

Support Layer

This layer includes configuration files, utility functions, environment settings, CORS configuration, and logging support. It ensures the application can be configured, monitored, and run reliably in different environments.

High-Level Architectural Principle

The system is designed so that each layer has a distinct role:

- frontend handles interaction
- backend handles coordination
- services handle logic
- support modules handle configuration and infrastructure concerns

This architecture makes the application easier to scale and update. For example, the LLM provider or vector database logic can be changed without redesigning the user interface.

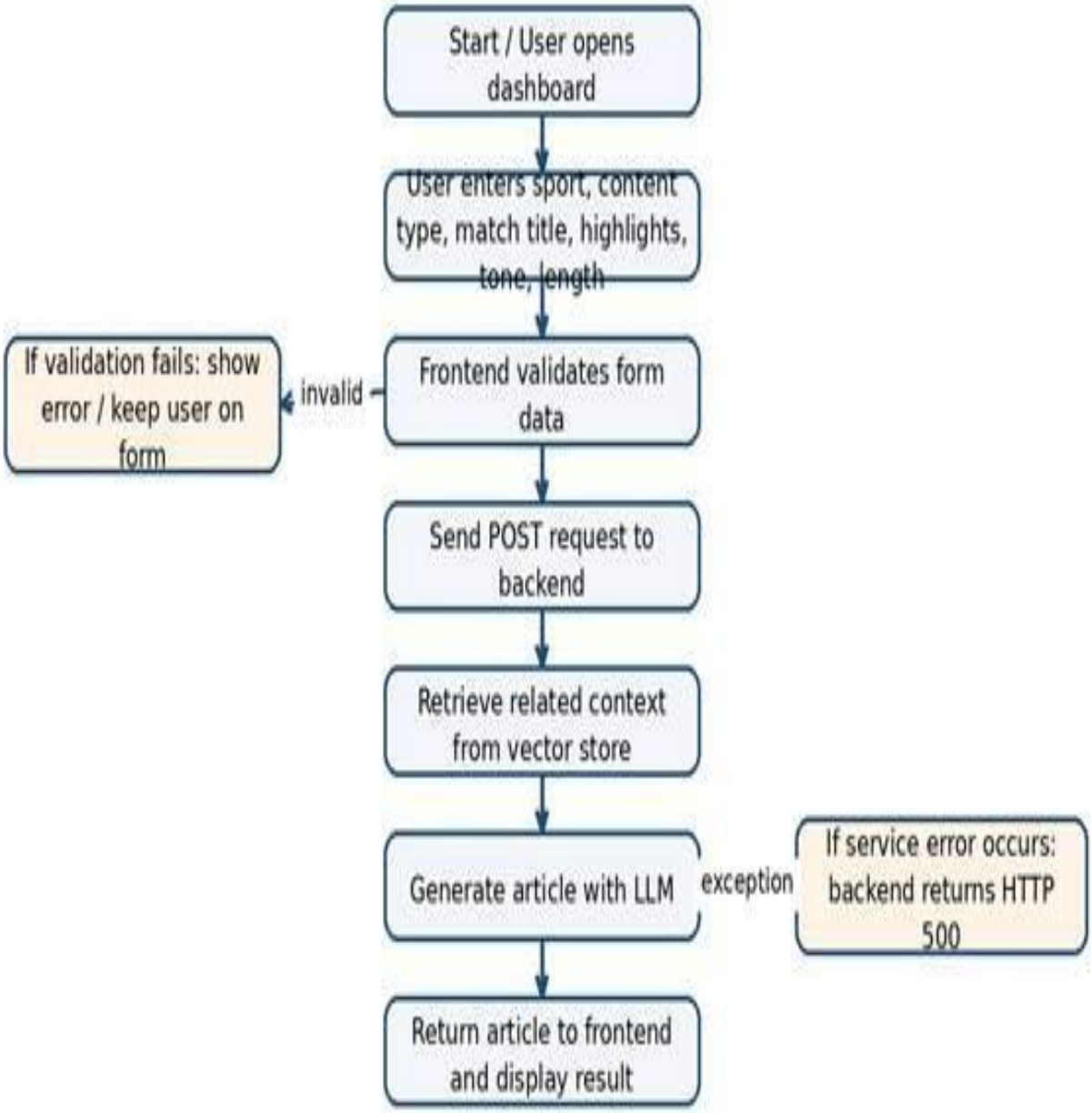
2.2 Process Flow

The overall process flow of Sports_GenAI begins when the user accesses the application dashboard and enters the content generation details. The process is as follows:

1. The user opens the dashboard in the browser.
2. The user enters values such as sport, content type, match title, highlights, tone, and length.
3. The frontend performs basic client-side validation to ensure that required inputs are present and properly formatted.
4. After validation, the frontend sends an HTTP POST request to the backend API endpoint.
5. The FastAPI backend receives the request and validates it against the request schema.
6. The backend passes the relevant query or content signals to the vector retrieval service.

7. The vector service searches the FAISS-based vector store using embeddings and retrieves the most relevant sports context.
8. The backend forwards the user input and retrieves context to the prompt builder.
9. The prompt builder creates a structured prompt in a sports-journalism style.
10. The LLM service sends this prompt to the language model.
11. The language model generates the sports content.
12. The backend wraps the generated result in a JSON response.
13. The frontend receives the response and displays the article to the user.

Process Flow / Activity Diagram



2.3 Information Flow

Information in the Sports_GenAI application moves in a sequential request-response pattern.

Input Information Flow

The user provides structured data through the frontend. This data typically includes:

- sport
- content type
- match title
- highlights
- tone
- length

This information flows from the UI form to the backend API in JSON format.

Backend Information Flow

- Once received by the backend:
- the request data is parsed and validated
- a search query is formed for contextual retrieval
- the vector service retrieves context
- the prompt builder combines user input and context
- the LLM service generates a textual response

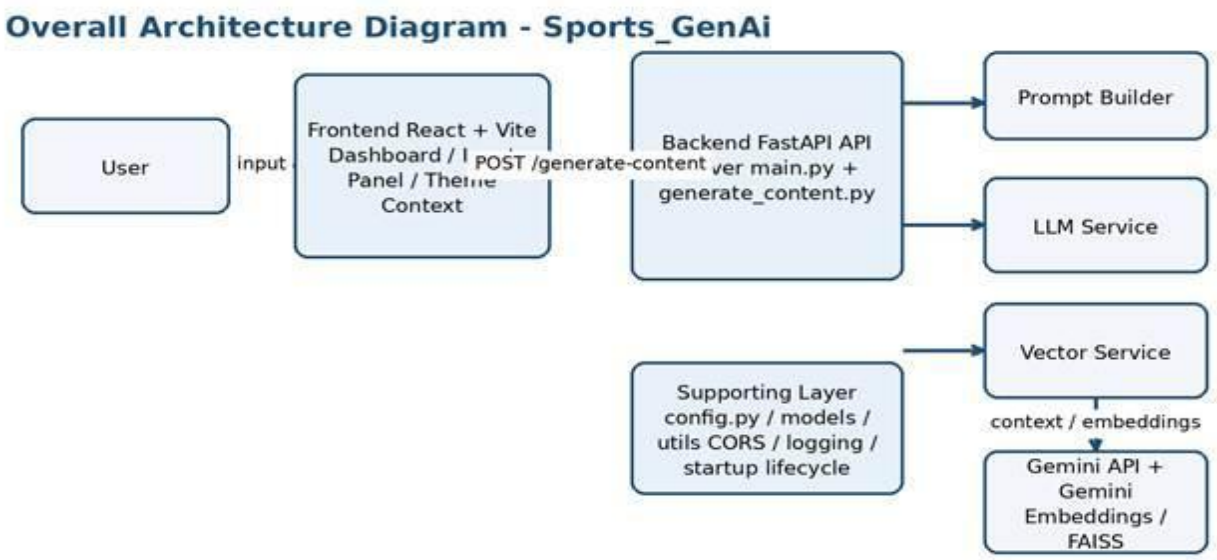
Output Information Flow

The generated content is returned as a backend response, generally in a JSON structure that includes:

- success flag
- generated article text
- optional error message in failure cases

High-Level Sequence

User → Dashboard → API Route → Vector Service → Prompt Builder → LLM Service → API Response → Dashboard



2.4 Components Design

1. Frontend Component

- dashboard page
- input form
- buttons and controls for submission
- result display area
- loading and error states

Responsibilities:

- collect user input
- validate mandatory fields
- call backend APIs
- display the response

2. API Layer Component

This component exposes REST endpoints such as:

- **/generate-content**
- **/health**

Responsibilities:

- accept requests
- perform schema validation
- route requests to internal services
- return standardized responses

3. Request Model Component

This component defines the structure of incoming data and ensures data consistency.

Responsibilities:

- field-level validation
- data typing
- request schema enforcement

Typical fields include those listed in the LLD data model: sport, content type, match title, highlights, tone, and length.

4. Prompt Builder Component

This component transforms structured inputs into a natural-language prompt suitable for the AI model.

Responsibilities:

- organize content in journalistic format
- include title, overview, key moments, performances, and conclusion
- combine retrieved context with user input

5. Vector Service Component

This component supports contextual enhancement through semantic retrieval.

Responsibilities:

- initialize or connect to the vector database
- create or use embeddings
- perform similarity search
- return relevant context to the backend

6. LLM Service Component

This component is responsible for the final text generation.

Responsibilities:

- receive final prompt
- invoke the selected language model
- process the response
- return generated article text

7. Configuration and Utility Component

This includes:

- environment variables
- logging
- CORS configuration
- helper functions
- application settings

Responsibilities:

- centralize configuration
- support maintainability

2.5 Key Design Considerations

Several design considerations were taken into account while designing Sports_GenAI.

Modularity

Each responsibility is assigned to a separate module. This makes the system easier to maintain and extend.

Separation of Concerns

The UI, API routing, context retrieval, prompt construction, and AI generation are logically separated. This reduces coupling and makes debugging easier.

Scalability

The architecture supports future enhancements such as:

- adding more content types
- replacing the language model
- moving from local vector storage to cloud vector database

Reusability

The prompt builder, vector search, and LLM services are reusable modules and can support future features such as match summaries, tournament reports, and player analysis.

Reliability

The backend is expected to use exception handling and proper API response codes so that system failures do not crash the application unexpectedly.

Usability

The user-facing design is simple and form-based, making it easy for non-technical users to operate the system.

Interoperability

The use of JSON and REST APIs allows easy integration between frontend and backend and supports future mobile or third-party integrations.

3. Data Design

3.1 Data Model

At a high level, the core data model in the system is the content generation request. The fields used by the system are:

Field Name	Description
sport	Represents the selected sport, such as cricket, football, tennis, or basketball.
content type	Represents the required output type, such as article, summary, report, or blog.
match title	Represents the title or subject of the match or event.
highlights	Contains important details, match events, or key points that should be reflected in the generated content.
tone	Defines the writing style, for example professional, exciting, analytical, or casual.
length	Specifies the desired size of output, such as short, medium, or detailed.

Response Model

The high-level response includes:

- success status
- generated article text
- optional error information

3.2 Data Access Mechanism

The Sports_GenAI system accesses data through multiple mechanisms depending on the type of data involved.

User Input Data Access

User input is collected in the frontend and transmitted to the backend through HTTP POST requests in JSON format.

Configuration Data Access

System configuration is read from application configuration files and environment variables. These may include API keys, model settings, path information, and CORS settings.

Context Retrieval Data Access

The vector service accesses contextual data through a vector search mechanism. The process generally includes:

- converting query text into embeddings
- searching the FAISS index
- retrieving the most similar entries
- returning that context to the prompt builder

Generated Output Access

Generated output is not permanently required for immediate operation unless logging or future storage is added. It is returned directly in the API response to the frontend.

3.3 Data Retention Policies

At the current project level, Sports_GenAI is primarily a real-time generation system and may not require long-term storage of user-entered content or generated results unless explicitly implemented.

A suitable high-level retention policy for this project is:

- user input data is retained only for the duration of request processing
- generated output is displayed to the user and not permanently stored by default

- logs may be stored temporarily for debugging and monitoring
- configuration and model-related files remain stored as part of deployment assets
- vector data remains available as long as the retrieval module requires it

If future versions introduce user accounts or content history, retention policies should be updated to define:

- how long user requests are stored
- whether generated content history is retained
- how deletion or archival will be handled
- privacy rules for stored data

3.4 Data Migration

In the current scope, large-scale data migration is minimal because the application is service-oriented and does not appear to rely on a traditional transactional database as its primary store.

However, high-level migration scenarios may include:

- migrating vector data from one FAISS index to another
- changing embedding models and rebuilding the vector index
- moving configuration from local files to cloud-based secret management
- shifting generated logs or metadata to persistent storage
- migrating deployment environments from local setup to containers or Kubernetes

If the system is upgraded in the future, migration should follow these steps:

1. back up existing vector and configuration data
2. validate the new environment
3. rebuild or import embeddings as needed
4. test API compatibility
5. verify content quality after migration

4. Interfaces

The Sports_GenAI system includes both user-facing and system-facing interfaces.

User Interface

The primary user interface is a web dashboard. It provides:

- input controls for generation parameters
- submit button to request content
- area to view generated sports content
- loading indicators
- error feedback

4.1 API Interface

Endpoint	Method	Input	Output	Purpose
/Generate-content	POST	sport, content type, match title, highlights, tone, length	success, generated article, error (optional)	Accepts user input, performs context retrieval and AI generation, returns article
/Health	GET	None	status (running/not running)	Checks backend health, useful for monitoring and debugging

4.2 Internal Service Interfaces

Interaction	Description
Route Layer → Prompt Builder	Converts user input into a structured prompt suitable for AI generation
Route Layer → Vector Service	Retrieves relevant sports context using FAISS vector database
Route Layer → LLM Service	Sends prompt to LLM and receives generated sports content

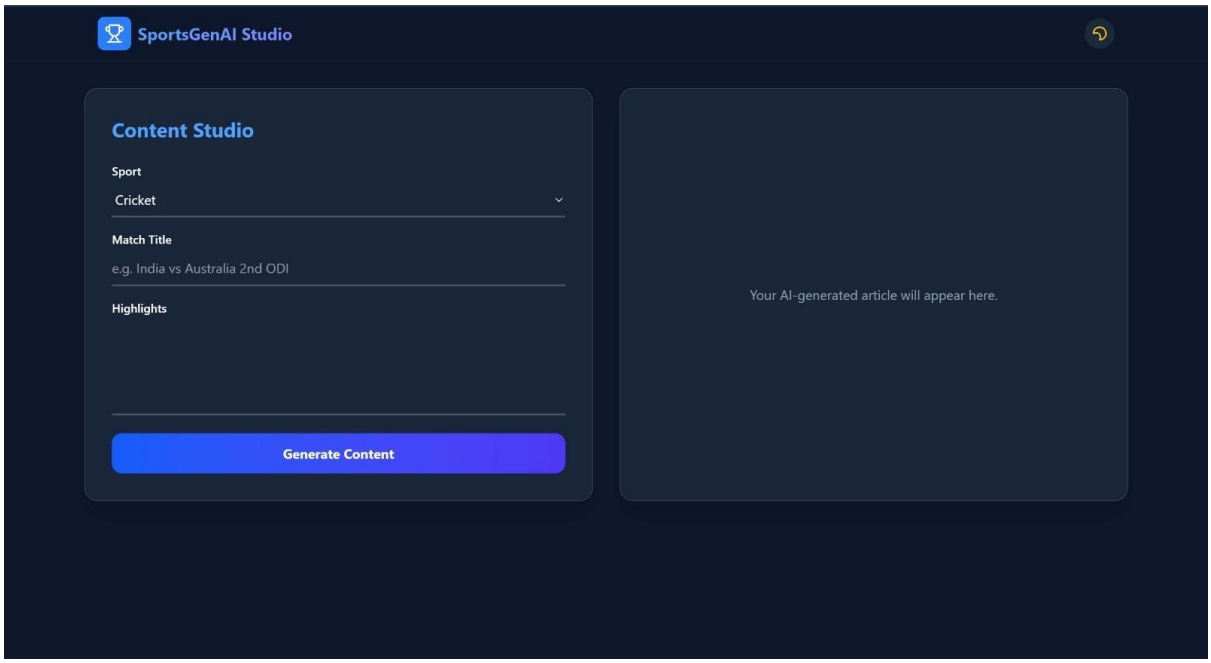
4.4 External Interfaces

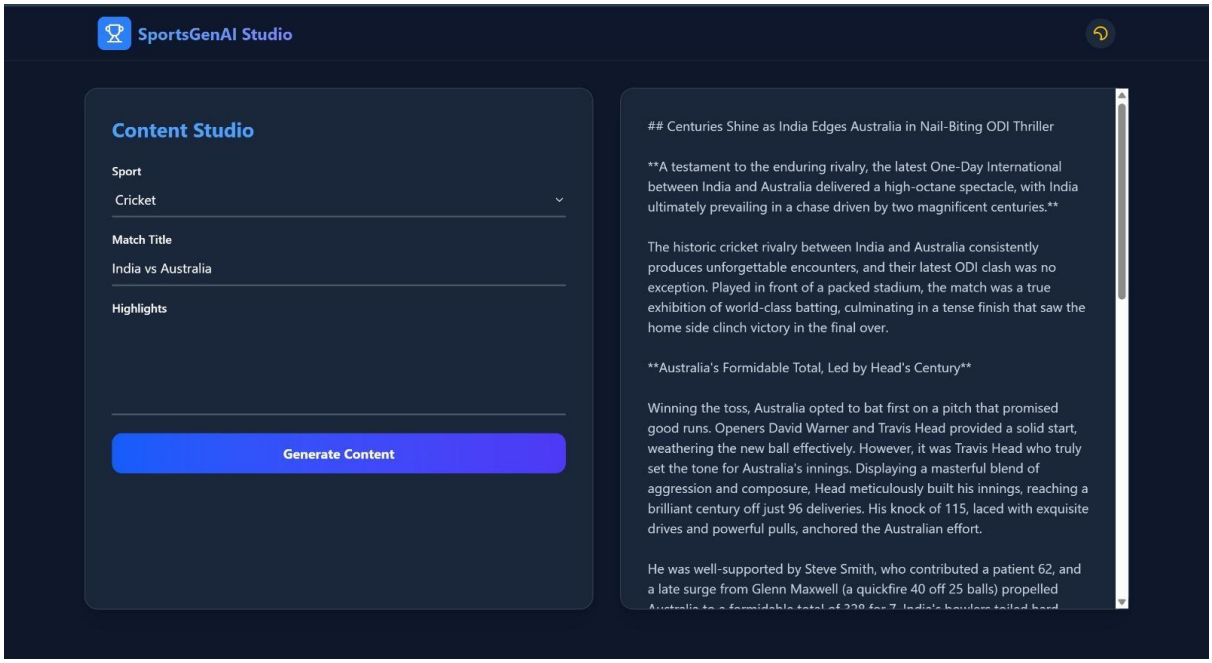
External System	Description	Purpose
LLM Provider (Gemini API)	External generative AI service used to generate sports content	Processes prompt and returns natural language article output
Embedding Service	Service used to convert text into vector embeddings	Enables semantic search and similarity matching for context retrieval
FAISS Vector Database	Local or integrated vector storage system for embeddings	Stores and retrieves relevant sports context using similarity search
Deployment & Monitoring Tools	Tools such as Docker, Kubernetes, or monitoring dashboards (future scope)	Used for deployment, scaling, logging, and system monitoring

5. State and Session Management

The Sports_GenAI application is primarily stateless at the backend level.

- **screenshots**





Stateless Request Handling

Each generation request is processed independently. The backend does not need to maintain long-running user session state to generate content. This improves scalability and simplifies deployment.

Frontend State Management

The frontend may maintain temporary state for:

- form values
- loading status
- API response data
- error messages
- theme selection or UI preferences

This state exists only during the user's interaction with the page and is generally managed in the browser.

Session Management

At the current scope:

- no user authentication is mandatory
- no persistent login session is required
- no role-based access control is necessary for basic functionality

If future versions add user accounts, session management may include:

- token-based authentication
- session expiration
- user-specific generation history
- access control policies

6. Caching

Caching in the current project may be minimal, but it can be described at a high level as follows.

Frontend Caching

The browser may cache static frontend assets such as JavaScript bundles, CSS, and images.

Backend Caching

Backend-level caching can be introduced for:

- repeated vector retrieval results
- repeated prompt templates
- static metadata or supported sport types
- health check optimizations

Model and Retrieval Optimization

The system may keep:

- loaded vector indices in memory
- initialized embedding or LLM clients ready for reuse
- commonly used configuration objects cached during runtime

Current Design Position

For an academic prototype, caching may not be heavily implemented, but the architecture supports adding caching later to improve performance under higher request loads.

7. Non-Functional Requirements

7.1 Security Aspects

Security in Sports_GenAI should focus on protecting the backend service, API access, and sensitive configuration values.

Input Validation

All user inputs must be validated at both frontend and backend levels to reduce malformed requests and improve reliability.

API Security

If deployed publicly, the API should be protected using:

- authentication or API keys
- rate limiting
- proper request validation
- restricted CORS policies

Environment Protection

Sensitive values such as model API keys and configuration secrets should be stored in environment variables and never hardcoded into source code.

Error Handling

The application should avoid exposing internal stack traces or confidential details in user-visible error messages.

CORS Configuration

Cross-origin access should be restricted to trusted frontend origins in production.

Future Security Enhancements

Possible future additions include:

- HTTPS enforcement
- user authentication
- audit logging
- request throttling

7.2 Performance Aspects

Performance is important because the user expects generated content within an acceptable response time.

Key Performance Factors

- frontend responsiveness
- backend request processing speed
- vector search efficiency
- prompt generation efficiency
- LLM response time

Design Support for Performance

The current architecture helps performance through:

- modular services
- fast API handling with FastAPI
- efficient similarity retrieval using FAISS
- stateless request processing

Performance Goals

At a high level, the system should aim to:

- respond quickly to valid requests
- minimize unnecessary backend computation
- handle concurrent requests without crashing
- maintain acceptable latency during AI generation

Future Performance Improvements

- asynchronous request handling
- queue-based processing for heavy loads
- response caching
- optimized prompt length

8. References

The following sources are relevant to the design of the Sports_GenAI project:

1. Uploaded LLD document for Sports_GenAI, including architecture, process flow, information flow, components, design considerations, API catalogue, and data model.
2. frontend dashboard module
3. backend FastAPI main application
4. backend route for content generation
5. prompt builder service
6. vector retrieval service
7. LLM service
8. configuration and utility modules